

FC7xxx MSC Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/10/20	Initial release for MCAL V0.3.0

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules.....	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory.....	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area.....	9
Chapter 5 Interrupt Service Routine (ISR).....	10
Chapter 6 Error Report.....	11
6.1 Det.....	11
6.2 Dem.....	12
Chapter 7 Function Calls.....	13
7.1 Function Calls during Startup	13
7.2 Function Calls during Shutdown	13
7.3 Function Calls during Wake-up	13
Chapter 8 Other Requirements	14
8.1 Notification, Callback, Callout	14
8.2 Macros	14
Chapter 9 Integration Steps	15

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the MSC module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for MSC module
- Port: This module provides the port configurations if want output MSC signal through pin.
- Common: This module is the basic module which used to choose the chip.
- Ecuc: is required for selecting the reference to the partition.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)
- Dem: is required for signaling the production error detection (hardware failure, etc).
- Pwm: is required for generate pwm signal.
- Trgsel: is required for routing pwm signal to msc mudule.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- Dem: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the MSC module, the user needs to initialize the MCU module first
- Port: To output MSC signals, the user needs to initialize PORT module first and assign the Msc pins.

2.2 Files Required for Compile

Msc module files:

- _MCAL/Src/Msc/Src/CDD_Msc.c
- _MCAL/Src/Msc/Src/CDD_Msc_Irq.c
- _MCAL/Src/Msc/include/CDD_Msc_LLD.c
- _MCAL/Src/Msc/include/CDD_Msc.h
- _MCAL/Src/Msc/include/CDD_Msc_LLD.h
- _MCAL/Src/Msc/include/CDD_Msc_Hw.h
- _MCAL/Src/Msc/include/CDD_Msc_MemMap.h
- _MCAL/Src/Msc/include/CDD_Msc_Reg.h
- _MCAL/Src/Msc/include/CDD_Msc_version.h
- _MCAL_generate/src/CDD_Msc_PBcfg.c
- _MCAL_generate/src/CDD_Msc_Cfg.c
- _MCAL_generate/include/CDD_Msc_PBcfg.h
- _MCAL_generate/include/CDD_Msc_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Msc.h

Dem module files:

- Dem.h

2.3 Add Plug-ins

MSC module plug-ins are developed for EB tresos Studio, so, to use MSC plug-ins on the EB tresos Studio, the user needs to add the MSC module to the EB plug-ins folder first.

- 1) Copy the MSC module(_MCAL/EB_Plugins/eclipse/plugins/Msc) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the MSC module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the MSC module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
MSC_START_SEC_CONFIG_DATA_8 MSC_STOP_SEC_CONFIG_DATA_8 MSC_START_SEC_CONFIG_DATA_16 MSC_STOP_SEC_CONFIG_DATA_16 MSC_START_SEC_CONFIG_DATA_32 MSC_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
MSC_START_SEC_CONFIG_DATA_UNSPECIFIED MSC_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
MSC_START_SEC_CONST_BOOLEAN MSC_STOP_SEC_CONST_BOOLEAN MSC_START_SEC_CONST_8 MSC_STOP_SEC_CONST_8 MSC_START_SEC_CONST_16 MSC_STOP_SEC_CONST_16 MSC_START_SEC_CONST_32 MSC_STOP_SEC_CONST_32 MSC_START_SEC_CONST_UNSPECIFIED MSC_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
MSC_START_SEC_CODE MSC_STOP_SEC_CODE MSC_START_SEC_RAMCODE MSC_STOP_SEC_RAMCODE MSC_START_SEC_CODE_AC MSC_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
MSC_START_SEC_VAR_NO_INIT_BOOLEAN MSC_STOP_SEC_VAR_NO_INIT_BOOLEAN MSC_START_SEC_VAR_NO_INIT_8 MSC_STOP_SEC_VAR_NO_INIT_8 MSC_START_SEC_VAR_NO_INIT_16 MSC_STOP_SEC_VAR_NO_INIT_16 MSC_START_SEC_VAR_NO_INIT_32 MSC_STOP_SEC_VAR_NO_INIT_32 MSC_START_SEC_VAR_NO_INIT_UNSPECIFIED MSC_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
MSC_START_SEC_VAR_INIT_BOOLEAN MSC_STOP_SEC_VAR_INIT_BOOLEAN MSC_START_SEC_VAR_INIT_8 MSC_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These

Section Name	Section Type	Description
MSC_START_SEC_VAR_INIT_16 MSC_STOP_SEC_VAR_INIT_16 MSC_START_SEC_VAR_INIT_32 MSC_STOP_SEC_VAR_INIT_32 MSC_START_SEC_VAR_INIT_UNSPECIFIED MSC_STOP_SEC_VAR_INIT_UNSPECIFIED		variables are initialized by startup code (data).

Chapter 4 Exclusive Area

MSC module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the MSC driver:

- CDD_Msc.c:
 - MSC_EXCLUSIVE_AREA_11 is used in Msc_GetRxStatus.
 - MSC_EXCLUSIVE_AREA_12 is used in Msc_GetRxStatus16BitFrame.
 - MSC_EXCLUSIVE_AREA_13 is used in Msc_GetRxStatus16BitFrame.
 - MSC_EXCLUSIVE_AREA_14 is used in Msc_GetRxStatus.
 - MSC_EXCLUSIVE_AREA_15 is used in Msc_GetRxStatus16BitFrame.

- CDD_Msc_LLD.c:
 - MSC_EXCLUSIVE_AREA_00 is used in Msc_LLD_Init.
 - MSC_EXCLUSIVE_AREA_01 is used in Msc_LLD_DeInit.
 - MSC_EXCLUSIVE_AREA_02 is used in Msc_LLD_SetDataFrame.
 - MSC_EXCLUSIVE_AREA_03 is used in Msc_LLD_SetDataFrame.
 - MSC_EXCLUSIVE_AREA_04 is used in Msc_LLD_StartChannel.
 - MSC_EXCLUSIVE_AREA_05 is used in Msc_LLD_StopChannel.
 - MSC_EXCLUSIVE_AREA_06 is used in Msc_LLD_SendCommandFrame.
 - MSC_EXCLUSIVE_AREA_07 is used in Msc_LLD_GetTxStatus.
 - MSC_EXCLUSIVE_AREA_08 is used in Msc_LLD_GetRxStatus.
 - MSC_EXCLUSIVE_AREA_09 is used in Msc_LLD_GetRxStatus.
 - MSC_EXCLUSIVE_AREA_10 is used in Msc_LLD_GetRxErrorStatus.
 - MSC_EXCLUSIVE_AREA_16 is used in Msc_LLD_SwitchSDIChannel.
 - MSC_EXCLUSIVE_AREA_17 is used in Msc_LLD_SwitchSDIChannel.
 - MSC_EXCLUSIVE_AREA_18 is used in Msc_LLD_SwitchEN.
 - MSC_EXCLUSIVE_AREA_19 is used in Msc_LLD_SwitchEN.
 - MSC_EXCLUSIVE_AREA_20 is used in Msc_LLD_SwitchEN.
 - MSC_EXCLUSIVE_AREA_21 is used in Msc_LLD_SwitchEN.
 - MSC_EXCLUSIVE_AREA_22 is used in Msc_LLD_SwitchEN.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
MSC0	MSC0_IRQHandler	140
MSC1	MSC1_IRQHandler	141

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Msc_Init	MSC_E_STATE_TRANSITION; MSC_E_INVALID_POINTER; MSC_E_ALREADY_INITIALIZED;
Msc_DeInit	MSC_E_STATE_TRANSITION;
Msc_StartChannel	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_CHANNEL_ACTIVE;
Msc_SetDataFrame	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_INVALID_POINTER; MSC_E_CONFIG_VALUE;
Msc_SendDataFrame	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_CHANNEL_NOT_STARTED; MSC_E_INVALID_POINTER; MSC_E_CONFIG_VALUE;
Msc_StopChannel	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_CHANNEL_NOT_STARTED;
Msc_SendCommandFrame	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_TX_BUSY; MSC_E_CHANNEL_NOT_STARTED; MSC_E_INVALID_POINTER; MSC_E_PARAM_VALUE;
Msc_GetTxStatus	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL;
Msc_GetRxStatus	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_INVALID_POINTER;
Msc_GetRxStatus16BitFrame	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_INVALID_POINTER; MSC_E_INVALID_RXINDEX;
Msc_SwitchSDIChannel	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_RX_BUSY;

Msc_SwitchEN	MSC_E_UNINIT; MSC_E_INVALID_CHANNEL; MSC_E_TX_BUSY;
Msc_GetVersionInfo	MSC_E_INVALID_POINTER;

6.2 Dem

Function Name	Error Type
Msc_Init	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Msc_Delnit	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Msc_StartChannel	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Msc_StopChannel	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Msc_SwitchSDIChannel	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;
Msc_SwitchEN	DEM_EVENT_STATUS_FAILED; DEM_EVENT_STATUS_PASSED;

Chapter 7 Function Calls

7.1 Function Calls during Startup

MSC shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is `Msc_Init()`. The MCU module should be initialized before the MSC is initialized. The MSC module shall be initialized by `Msc_Init(<&Msc_Configuration>)` service call during the start-up before the MSC peripherals are used. Please note that GPIO pins used for connection of MSC physical layer have to be properly assigned to desired MSC module prior to the MSC initialization: so the MCU and PORT modules shall be initialized before MSC is initialized. After the MSC module is initialized, each MSC channel has to be initialized as well before using it. This is also done by the `Msc_Init(<&Msc_Configuration>)` service.

7.2 Function Calls during Shutdown

There is no shutdown specific procedure for Msc driver. MSC driver can stop transmission by using the following API services:

- `Msc_StopChannel(MSC_CHANNEL)`: which stop the transmission on MSC TX.

7.3 Function Calls during Wake-up

There is no shutdown specific procedure for Msc driver. MSC driver can start transmission by using the following API services:

- `Msc_StartChannel(MSC_CHANNEL)`: which start the transmission on MSC TX.

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification
None
- Callback
There are no callback functions within the MSC driver.
- Callout
There are no callout functions within the MSC driver.

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:
AUTOSAR_OS_NOT_USED needs to be defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to the ISR function, for example:

```
extern ISR(Msc_Isr_MSC_0);  
  
void MSC0_IRQHandler(void)  
{  
  
    Msc_Isr_MSC_0 ();  
  
}
```

- If OS is used:
Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the MSC module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the MSC module with dependent modules.